will ask for the server's root password during set-up. Provide the same and keep it secure.

The storage engines available in MariaDB 10.1

Storage engines form the most important part of MySQL and MariaDB. They decide how and where the data will be stored and provide features for data management. There are many storage engines supported by MariaDB, but we will discuss only the ones that are most commonly used.

Transactional engines: InnoDB, which originally came with MySQL, is the transactional engine by default for MySQL. MariaDB has a fork known as XtraDB as the default transactional engine; XtraDB is developed and maintained by Percona. XtraDB is also available as part of the Percona server. InnoDB/XtraDB is the most used ACID-compliant engine. It is safe and good for most use cases.

InnoDB is not known to be easy to backup. If you are using XtraDB, there is a tool from Percona known as *xtrabackup* that can be used for backing up XtraDB data. The use of this tool is related to the way data is stored by the engine. One can always use the good old *mysqldump* utility but it strains the CPU, as every row has to be translated into proper SQL statements so that it can be reloaded at the backup server – and the SQL statements have to be translated back to real data and also recreate any indices. On small size databases *mysqldump* is sufficient but for larger ones it is better to use *xtrabackup*. Replication to another server and taking backups there is better, too.

There is yet another ACID-compliant transactional engine present in MariaDB, called TokuDB. It is designed for storing large amounts of data in smaller spaces – up to 25x compression can be achieved. The typical use cases for TokuDB are applications where a lot of data is queried and updated at the same time and in environments where the data that is being worked upon at a given instant cannot fit in the RAM.

Non-transactional engines: MyISAM is the oldest engine in MySQL. MariaDB has an improved version – Aria. It has many performance related fixes and better safety, which is lacking in MyISAM. One disadvantage of MyISAM is that it locks the table when there is an update or insert going on – MyISAM does not support MVCC while Aria has it in its roadmap but is not there yet.

There are two other interesting storage engines – SphinxSE and MEMORY. SphinxSE can be used when full text search is needed – it relies on the external Sphinx search service. The MEMORY engine is useful for storing data purely in the RAM – there is no persistence of the data to disk and it will be lost when the server is restarted. Throwaway cache data is an ideal candidate to be stored in the MEMORY engine.

There are more storage engines supported by MariaDB such as Federated and Cassandra, which are not covered here. Information about the same is available at *mariadb.com*.

Performance tuning

Database performance tuning is a vast topic and depends highly on the workload of the application. Yet, there are some common principles that can be applied to every case. To begin with, check the hardware of your server. Databases require a lot of RAM and fast disk IO. The more the RAM and the faster the disk, the happier your database server is. Tools such as *PHPMyAdmin* (Web application) and *mysqltuner* (command line application) provide insights on server performance and suggested parameter tuning to achieve better performance.

It is highly recommended to have storage such as SSD in RAID 10 mode for achieving high throughput with a large database system. Just switching from a traditional rotational hard drive to SSD can improve the response times of the server significantly.

To achieve good throughput from a database, it is very important to design the database schema and use optimal queries. For example, if you have a table with a user's details such as name, email address, user name and you are running a query which searches for the user name without having an index on the user name field, then the performance will be bad because the database server has to do a full table scan for every such query.

A full table scan is one in which all the rows have to be scanned before arriving at a result. Badly designed queries, especially joins, can cause severe performance hits. MySQL and MariaDB use the nested loop algorithm for computing the result of joins. A nested loop algorithm works something like this: let us say you have three tables (t1, t2, t3) for which a combined result is required. First, the optimiser decides the driving table, and the next table in the join is queried based on results received from the driving table. This result set forms the base, using which the third table's data is matched. In mathematical terms, this is known as a Cartesian product.

Use caching

MySQL and MariaDB have a query cache, which can provide a significant boost in performance. It is useful in cases with less writes and mostly reads. If there are a lot of writes happening, then the server will be spending more time managing the cache instead of working on queries. For this reason it is not recommended to have a large size for the built-in MySQL query cache. Up to 512MB is sufficient. For more control the application should use its own caching such as Memcached or Redis.

Use EXPLAIN

When designing queries for the system, it is important to have optimal queries. MySQL will provide details of the execution plan of a query given to EXPLAIN. Optimizer trace is also available, which can be used to explore why one plan was chosen instead of the other. Optimizer trace is used by MySQL internally, but can be used while designing queries as well.

To use optimizer trace, type:

```
mysql> SET optimizer_trace="enabled=on"
```